

Week 1 · Day 2 — From Voltage to Meaning

Date: Wednesday, 16 September 2026 **Tracker position:** Stage 1 — Foundations of Computation · **Week 1: Electricity, States, Bits, and Information** **Today's target time:** ~2 hours (≈45 min read, ≈75 min hands-on)

Day 1 (15 Sep) introduced Week 1 conceptually in conversation. Today's lesson is the permanent written version, pushed deeper: we quantify *why* binary wins, define *information* precisely, and then **observe on real Linux** that one bit pattern genuinely carries four or five different meanings at once.

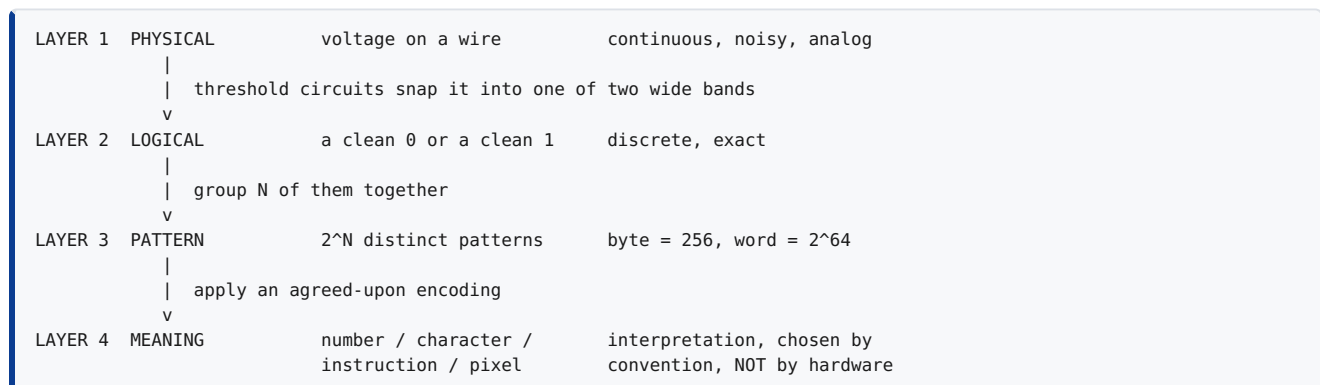
1. Where this sits

Week 1 Learning Objectives (from the tracker):

1. Understand voltage as a range rather than a perfect 0/1 value
2. Explain why digital systems use stable state ranges
3. Define bit, byte, word, encoding, and information
4. Convert values between decimal, binary, and hexadecimal

Objectives 1 and 2 are the *physics* half. Objectives 3 and 4 are the *representation* half. Today we finish both halves and prove the payoff on the machine.

2. The one diagram for this week



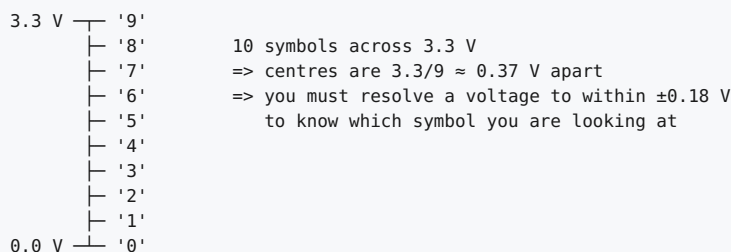
Everything in Week 1 is about understanding that these are four genuinely different layers, and that **the hardware only ever implements Layers 1-3**. Layer 4 exists only as an agreement between pieces of software.

3. Objective 1 & 2 — why two states, quantified

Yesterday's claim was "binary maximizes noise margin." Let's put numbers on it, because "noise margin" is not hand-waving — it's the entire reason digital logic works.

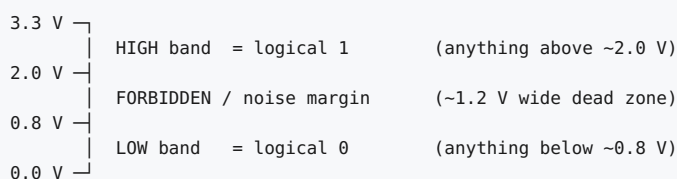
Take a chip with a 3.3 V supply rail.

Option A — encode 10 symbols directly (one voltage band per decimal digit)



Real chips routinely see tens to hundreds of millivolts of noise from crosstalk between adjacent wires, power-supply ripple (di/dt droop when billions of gates switch at once), and thermal noise. A ± 0.18 V budget is of the same order as the noise itself — so symbols get misread constantly. You would also need a 10-level analogue comparator at the receiving end of every single wire.

Option B — encode 2 symbols (binary)



Now noise has to move a signal across a **~ 1.2 V** dead zone to corrupt a bit — roughly an order of magnitude more than the noise present. And the receiving circuit is no longer an analogue comparator; it is just a transistor pair that is either on or off.

The trade-off, stated precisely: binary gives up *information density per wire* (1 bit instead of ~ 3.3 bits per wire) and buys back *enormous noise immunity* plus *drastically simpler switching elements*. Since we can trivially add more wires (parallel buses) or more time (serial clocking), but we cannot trivially make physics quieter, this is an overwhelmingly good trade.

This is engineering, not mathematics. Multi-level cell (MLC/TLC/QLC) NAND flash deliberately makes the opposite trade — it stores 2, 3, or 4 bits per cell using multiple charge levels, accepting far worse error rates, and then pays for it with heavy error-correcting codes. That is precisely why QLC SSDs are cheap per gigabyte but wear out faster and are slower to write. Same trade-off, different answer, because the constraints differ.

Why? question to sit with: if noise margin is so valuable, why have CPU supply voltages *fallen* over the decades (5 V $\rightarrow$ 3.3 V $\rightarrow$ ~ 1 V), shrinking the margin? (Hint: dynamic power $\approx C \cdot V^2 \cdot f$.)

4. Objective 3 — the five words, defined precisely

Term	Precise meaning	Common sloppy version to avoid
Bit	One binary digit: the resolution of exactly one two-way choice. The atomic unit of <i>distinguishability</i> .	"a bit is a 0 or 1 stored somewhere" — true but hides that it is a <i>choice being resolved</i>
Byte	8 bits grouped = $2^8 = 256$ distinct patterns. A <i>convention</i> (IBM System/360 onward), not a law.	"a byte is a character" — only true for ASCII-era encodings; UTF-8 characters are 1–4 bytes
Word	The CPU's natural operand/register width — 64 bits on x86-64. Architecture-defined.	"a word is 2 bytes" — that is only the x86-16 legacy meaning that survives in the name of the <code>word</code> / <code>dword</code> / <code>qword</code> assembler directives
Encoding	The agreed rule mapping bit patterns $\leftrightarrow$ meanings. Lives in software/convention, never in the wires.	"the data is encoded in binary" — conflates Layer 3 with Layer 4
Information	The reduction of uncertainty. One bit of information = one previously-open two-way question now settled.	"information is data" — data is the pattern; information is what learning it <i>resolved</i>

The subtle one: information ≠ bits-of-storage

A bit of *storage* is a container. A bit of *information* is what you actually learn by looking at it.

- A wire hard-wired to ground is always 0. Reading it teaches you nothing: it carries **0 bits of information**, despite occupying 1 bit of storage.
- A fair coin flip stored in that bit carries the full **1 bit of information**.
- A biased coin (99% heads) carries only ≈ 0.08 bits — you could already guess it.

This gap between *storage capacity* and *information content* is exactly what every compression algorithm exploits, and it is the seed of Shannon's information theory. You do not need the mathematics this week; you need the distinction.

5. Objective 4 — conversions, and why hex exists at all

The mechanical part

Decimal → binary: repeatedly divide by 2, collect remainders bottom-up. Decimal → hex: repeatedly divide by 16. You will drill this 25 times today.

The part actually worth understanding

Why does hexadecimal map cleanly onto binary while decimal does not?

Because $16 = 2^4$ exactly. So one hex digit $\equiv$ exactly four bits, always, with no carry leaking across the boundary:

```
0xF  A  D  E
|    |  |  |
1111 1010 1101 1110    <- each hex digit is an independent 4-bit nibble
```

Ten, by contrast, is not a power of two. A decimal digit boundary lands *in the middle* of bit patterns, so you cannot look at any fixed group of bits and read off one decimal digit:

```
decimal 500 = 111110100 binary
the digit '5' does not correspond to any contiguous slice of those bits
```

That is the whole reason systems programmers read memory dumps, register values, error codes, and page-table entries in hex: **hex is a human-readable shorthand for binary**, whereas decimal is a translation that destroys the bit structure. Recall yesterday's page-fault error code — error 6 was readable only once we wrote it as `0b110` and split it into bit positions. Hex/binary preserve bit structure; decimal hides it.

6. OBSERVE ON REAL LINUX — one pattern, five meanings

This is today's key lab, and it directly settles the Week 1 mastery checkpoint. Every command and every output below was run on this machine — reproduce it yourself.

Step 1 — put four specific bytes on disk

```
cd /tmp
printf 'FADE' > bits_demo.bin
xxd bits_demo.bin
```

```
00000000: 4641 4445                                FADE
```

So the file holds the bit pattern:

```
0x46      0x41      0x44      0x45
01000110  01000001  01000100  01000101
```

Those 32 bits are now fixed. Nothing about them will change for the rest of this lab. Only our *interpretation* changes.

Step 2 — interpretation A, B, C: bytes, hex, characters

```
od -An -tu1 -tx1 -tc bits_demo.bin
```

```
70 65 68 69      <- as unsigned 8-bit integers
46 41 44 45      <- as hexadecimal
F  A  D  E       <- as ASCII characters
```

Step 3 — interpretation D: one 32-bit integer

```
od -An -tu4 bits_demo.bin
```

```
1162101062
```

The same 32 bits, read as a single little-endian unsigned 32-bit integer, are 1,162,101,062. (Why that value and not 0x46414445 = 1,178,944,069? Because x86 is little-endian — the *lowest* address holds the *least* significant byte. Endianness is formally Week 13; for now just register that byte order is yet another convention layered on the same bits.)

Step 4 — interpretation E: machine instructions

```
objdump -D -b binary -m i386:x86-64 bits_demo.bin
```

```
0000000000000000 <.data>:
 0: 46                rex.RX
 1: 41                rex.B
 2: 44                rex.R
 3: 45                rex.RB
```

The CPU's decoder reads those identical bytes as four **REX prefix** bytes — valid x86-64 instruction prefixes that modify which registers a following instruction can reach. (They are legal encodings, they just do nothing useful without a following opcode.) For contrast, a different 7-byte pattern decodes to a real instruction:

```
printf '\x48\xc7\xc0\x0a\x00\x00\x00' > insn.bin
objdump -D -b binary -m i386:x86-64 insn.bin
```

```
0: 48 c7 c0 0a 00 00 00    mov    $0xa,%rax
```

Step 5 — interpretation F: pixels

```
python3 -c "
for b in open('/tmp/bits_demo.bin','rb').read():
    print(''.join('#' if (b>>i)&1 else '.' for i in range(7,-1,-1)), f' <- 0x{b:02x}')
"
```

```
. # . . . # # .    <- 0x46
. # . . . . . #    <- 0x41
. # . . . # . .    <- 0x44
. # . . . # . #    <- 0x45
```

Read as a 1-bit-per-pixel bitmap, the same bytes are a small monochrome image (you can even see the shared . # . . . left edge, because all four ASCII capitals share high bits).

What just happened

One immutable 32-bit pattern on disk was simultaneously:

Interpretation	Value
four unsigned bytes	70, 65, 68, 69
hex dump	46 41 44 45
ASCII text	FADE
one LE uint32	1162101062
x86-64 instructions	four REX prefixes
1bpp bitmap	a 4-row pixel pattern

The bytes never changed. **The meaning was supplied entirely by the tool doing the reading.** That is Layer 4, demonstrated rather than asserted.

7. Hands-on work for today

1. **The 25 conversions**, by hand, no calculator: `~/masterclass/tracker/week01/conversions_exercise.txt` Then self-check: `python3 ~/masterclass/tracker/week01/check_conversions.py`
2. **Reproduce the five-interpretations lab** above yourself, start to finish, typing the commands rather than copy-pasting where practical.
3. **Rewrite** `~/masterclass/tracker/week01/why_binary_is_practical.md` in your own words. The tracker's standard is *explain it*, and the draft in that file is mine, not yours.
4. **Interactive challenge** (separate file, opens in a browser): `~/masterclass/daily/challenges/2026-09-16-bit-lab.html`

8. Mastery checkpoint

"Explain why the same bit pattern can represent a number, character, instruction, or pixel."

Target answer, in your own words, without notes: the hardware stores and moves only patterns of two-state signals; it has no notion of meaning. An encoding is an external agreement about how to interpret a pattern, so the *same* pattern yields a number, a character, an instruction, or a pixel depending on which agreement the reading code applies — as demonstrated above where `46 41 44 45` was all of those at once. Corollary worth stating: this is exactly why memory-safety bugs are dangerous — convincing a program to interpret attacker-supplied *data* bytes as *instruction* bytes is a Layer 4 confusion, not a Layer 1-3 failure.

9. Why? notebook — add these

- If noise margin is so valuable, why did supply voltages drop from 5 V to ~1 V?
- Why is a byte 8 bits and not 6, 9, or 12? (It has been all of those, historically.)
- QLC flash stores 4 bits per cell using 16 charge levels. What does it pay for that, and why is that trade acceptable there but not inside a CPU?
- `od -tu4` printed 1162101062, not 1178944069. What decided that?
- If a wire tied to ground carries 0 bits of information, how much information does a file of one million identical bytes carry — and what does that imply about compression?

10. Evidence checklist (tracker Week 1)

- [] Notes updated
- [] Diagram completed → `~/masterclass/tracker/week01/diagram.txt`
- [] Code / experiment committed → conversions checker + five-interpretations lab
- [] Results recorded (write down which conversions you got wrong, and why)

- [] Questions notebook updated (§9 above)
- [] One verbal explanation completed without notes (§8 above)

Fill in the tracker's own Week 1 row yourself — start/end date, hours, honest confidence /5, status. That self-assessment is yours to make.

Next lesson: Week 1 · Day 3 — finish Week 1 evidence, then open **Week 2: Signed Numbers and Bitwise Reasoning** (two's complement from first principles, overflow, masks, and why the kernel's bit-field macros look the way they do).